

Informe de la Pràctica: Web Apps & Cloud

Nerea de la Torre, 1669013 i Adrián Prego, 1672251

1. Introducció

En aquesta pràctica es treballa amb el desplegament d'una aplicació web utilitzant Flask en un entorn local i posteriorment en un entorn AWS EC2. A més, s'integrarà un servei de notifikacions web amb Firebase Cloud Messaging (FCM).

2. Objectius

- Comprendre el funcionament del Cloud Computing.
- Aprendre a desplegar una aplicació web en un entorn local i en AWS EC2.
- Gestionar serveis cloud amb AWS CLI.
- Integrar un sistema de notifikacions push utilitzant Firebase.

3. Fase 1: Desplegament local amb Flask

En aquesta fase, es prepara i es llança una aplicació web simple amb Flask (un framework de Python que ens serveix per construir un servidor web) en un entorn local. Aquest pas és essencial per provar el funcionament abans de desplegar la web al núvol.

Per evitar crear una aplicació des de zero, clonem un projecte exemple que ja conté una estructura bàsica de Flask.

3.1. Creació i activació de l'entorn virtual

L'ús d'un entorn virtual ens permet gestionar les dependències de Python de manera aïllada per evitar conflictes amb altres projectes o biblioteques del sistema.

Primer, des de la finestra de terminal en el nostre ordinador, creem l'entorn virtual `.venv` i després, l'activem per executar totes les comandes dins d'aquest entorn. Finalment, instal·lem les dependències necessàries des del fitxer `requirements.txt`.

3.2. Execució del servidor Flask

Amb l'entorn preparat, executem Flask amb `python` per posar en funcionament l'aplicació. L'aplicació s'executa a `http://127.0.0.1:5000/`, que és l'adreça local on podem accedir-hi des del navegador.

3.3. Preguntes i modificacions requerides

Si obrim el codi de l'aplicació web, veiem un exemple bàsic d'una aplicació web utilitzant Flask.

Pregunta

Quina funció té el decorador `@app.route("/")`?

El decorador `@app.route("/")` en Flask serveix per definir una ruta en la qual el servidor respondrà quan es faci una petició a la URL base (/).

Si obrim el navegador a `http://127.0.0.1:5000/`, es cridarà aquesta funció i es mostrarà el text "Hola, món!".

Pregunta

Modifiqueu el codi perquè la pàgina web tingui un fons blau clar i mostri el nom dels integrants del grup.

Hem modificat la funció del codi:

```
1 from flask import Flask
2 app = Flask(__name__)
3 @app.route("/")
4 def home():
5     return """
6     <!DOCTYPE html>
7     <html lang="ca">
8     <head>
9         <style>
10             body { background-color:
11                 lightblue; text-align: center; }
12             h1 { color: navy; }
13         </style>
14     </head>
15     <body>
16         <h1>Benvinguts a la nostra
17         pàgina!</h1>
18         <p>Aquests som els creadors:</p>
19         <ul>
20             <li>Adrián</li>
21             <li>Nerea</li>
22         </ul>
23     </body>
24     </html>
25     """
26 if __name__ == '__main__':
27     app.run(host='0.0.0.0', port=80)
```

Obtenim aquest resultat:

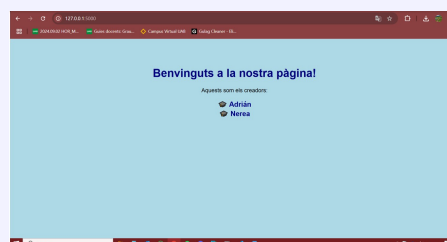


Figure 1. Pàgina web

Pregunta

Modifiqueu la funció perquè el servidor web escolti al port 8001.

Configurem Flask perquè escolti en el port 8001, el qual serà necessari en fases posteriors per compatibilitat amb el desplegament en AWS. Modifiquem aquesta secció del codi:

```
1 if __name__ == '__main__':
2     app.run(host='0.0.0.0', port=8001)
```

4. Fase 2: Desplegament en AWS EC2

En aquesta fase, l'objectiu és portar l'aplicació Flask al núvol mitjançant AWS EC2, un servei de computació en el núvol que permet l'execució de servidors virtuals.

4.1. Configuració d'AWS CloudShell i creació d'una instància EC2

Per començar, obrim la terminal AWS CloudShell, mitjançant **AWS CLI**, que és una eina de línia de comandes que permet interactuar amb els serveis d'AWS directament des de la terminal.

Després, es crea una **instància EC2**, que és un servidor virtual al núvol:

1. Primer especifiquem el tipus de màquina (**t2.micro** per aprofitar el nivell gratuït d'AWS).
2. Pel que fa al sistema operatiu, seleccionem una imatge **AMI** d'Amazon Linux, mitjançant la comanda:

```
1 aws ec2 describe-images --owners amazon --filters "Name=name,Values=amzn2-ami-hvm-*-x86_64-gp2" --query 'Images[*].[ImageId,Name,Description]' --output table
```

Escollim la imatge **ami-032ae1bcc5be78ca**.

Pregunta

Què fa exactament el paràmetre `-query` en aquesta comanda?

El paràmetre `-query` en aquesta comanda s'utilitza per **filtrar i formatar** la sortida de la resposta JSON que retorna AWS CLI. En aquest cas, la consulta `'Images[*].[ImageId,Name,Description]'` selecciona totes les imatges (`Images[*]`) i retorna només tres camps de cadascuna:

- (a) **ImageId**: ID de la imatge AMI
- (b) **Name**: Nom de la imatge.
- (c) **Description**: Descripció de la imatge.

Això fa que la sortida mostri només aquesta informació en lloc de tota la resposta JSON, fent-la més fàcil de llegir.

3. Creem un **grup de seguretat**, que en el nostre cas és **sg-0a0df2c7c479ee2fd**, que actua com un tallafoc i controla el tràfic d'entrada i sortida. Es realitzen dues accions essencials:
 - **Permetre connexions SSH (port 22)**: Aquesta comanda permet connexions SSH per administrar la instància de manera remota. El `-cidr 0.0.0.0/0` obre l'accés des de qualsevol IP, però en entorns segurs es recomana restringir-ho a IPs específiques.
 - **Permetre connexions HTTP (port 80)**: Aquesta comanda obre el port 80 per permetre l'accés HTTP a la web desplegada a la instància. Això fa possible que qualsevol usuari pugui veure l'aplicació des del seu navegador.
4. Creem una **clau SSH** per a la connexió segura, en el nostre cas és **practical1**.

Una vegada tenim tota aquesta informació, executem la màquina virtual EC2 amb la comanda següent:

```
1 aws ec2 run-instances --image-id ami-032ae1bcc5be78ca --count 1 --instance-type t2.micro --key-name practical1 --security-group-ids sg-0a0df2c7c479ee2fd
```

4.2. Connexió a la instància

Un cop l'instància està en funcionament, ens connectem a ella mitjançant SSH, utilitzant la clau generada prèviament i la IP pública de la instància, que en el nostre cas és **44.201.144.184**. Això ens permet tenir accés remot a la màquina per poder-hi instal·lar i executar l'aplicació.

```
1 ssh -i practical1.pem ec2-user@44.201.144.184
```

4.3. Instal·lació de dependències i configuració del servidor

Dins la instància EC2, primer ens hem d'assegurar de tenir instal·lats Python3 i Git, necessaris per executar Flask i clonar el codi de l'aplicació. Després, creem un entorn virtual i instal·lem les dependències (`requirements.txt`), tal i com hem vist a l'apartat 3.1.

A continuació, modifiquem l'aplicació `app.py` perquè escolti a **0.0.0.0:80**. Això permet que l'aplicació sigui accessible des de qualsevol navegador a través de l'adreça pública de l'instància.

4.4. Execució de l'aplicació i canvi del port

Després d'haver configurat la instància, s'executa el servidor Flask executant la nostra `app.py`. Així, iniciem l'aplicació Flask a la instància EC2, permetent accedir-hi des del navegador a través de l'adreça pública de la instància.

Pregunta

Modifiqueu el que creieu necessari perquè el servidor web escolti al port 8001.

Modifiquem `app.py` per escoltar en el port 8001:

```
1 app.run(host='0.0.0.0', port=8001)
```

Aquest canvi implica que el servidor ja no escoltarà en el port estàndard HTTP (80) sinó en el port 8001. Per tant, també serà necessari obrir aquest port en el Security Group d'AWS amb la següent comanda:

```
1 aws ec2 authorize-security-group-ingress --group-id sg-0a0df2c7c479ee2fd --protocol tcp --port 8001 --cidr 0.0.0.0/0
```

Amb això, qualsevol usuari podrà accedir a l'aplicació des del navegador utilitzant `http://44.201.144.184:8001`.

5. Integració amb Firebase per a notifikacions web

En aquesta secció es descriu el procés d'integració de **Firestore Cloud Messaging** o FCM (un servei propor-

cionat per Google que permet enviar notificacions web) en una aplicació web amb Flask, per tal d'implementar notificacions push. Aquesta funcionalitat permet enviar missatges als usuaris en temps real, independentment de si tenen l'aplicació oberta o no. Concretament la pràctica simula la comunicació entre dos usuaris (un en Firefox i l'altre en Chrome).

5.1. Descàrrega del projecte i estructura de l'aplicació

Primer hem descarregat el codi font del projecte, i hem extret l'arxiu ZIP que conté els fitxers necessaris per a la implementació de l'aplicació web.

Després d'extreure els arxius, ens assegurem de que l'estructura de la aplicació sigui la següent:

```
1 PushWebApp-main/  
2 -- static/ #arxiu JavaScript i estatics  
3 -- templates/ #arxiu HTML  
4 -- WebApp.py #arxiu controlador principal de  
   la aplicacio Flask
```

Vam haver de crear aquestes carpetes i moure els arxius a dins.

5.2. Configuració de Firebase y Ngrok

Per integrar Firebase Cloud Messaging (FCM), hem seguit aquests passos:

1. **Afegir credencials de Firebase:** Hem editat el fitxer `static/app.js` per incloure les credencials de Firebase necessàries per a permetre la comunicació entre el servidor de Firebase i l'aplicació web.

2. **Execució de l'aplicació:** Un cop configurat el arxiu `app.js`, hem posat en marxa l'aplicació Flask mitjançant `python3`. Aquest pas permet iniciar el servidor Flask que gestionarà les notificacions web. Fins aquí es pot veure la pàgina principal, però no es pot rebre notificacions fins que no es configuri el permís HTTPS.

3. **Configuració del permís HTTPS amb ngrok:** Firebase requereix un domini segur per a la recepció de notificacions push, no obstant això, configurar un certificat SSL en un entorn de proves pot ser complicat i costós. Però utilitzant **ngrok**, generem una URL segura, ja que crea un túnel segur cap al nostre servidor local, fent-lo accessible des d'internet amb HTTPS, sense necessitat de configurar un servidor amb certificat.

Primer de tot es descarrega i instal·la ngrok a la màquina local. A continuació, es genera un authtoken personal des del panell de control per configurar ngrok: `2uD3SFZAvq51s5...C7QgJgiScEDW`.

Amb això, fem la instal·lació i autèntiquem el nostre token en la màquina. Amb aquestes accions, es crea una comunicació segura entre el navegador i Firebase.

4. **Executar ngrok en el Port 80:** Abans de configurar el port de ngrok correctament, tornem a executar la nostra WebApp, però en aquesta ocasió en mode background. D'aquesta forma, ens permet tenir el servidor Flask obert sense que ens apareixi a la terminal explícitament, i poguem executar la comanda que generarà una URL pública (HTTPS), que estarà

connectada al nostre servidor local. Copiem la URL generada, i la posem al cercador per accedir a l'aplicació.

Aquest pas és essencial perquè els navegadors requereixen HTTPS per gestionar notificacions push. Com que el port 80 és el port estàndard per a HTTP, executar ngrok en aquest port permet que les sol·licituds HTTP siguin redirigides de manera segura.

5.3. Execució de l'aplicació i prova de notificacions

Per últim validarem que l'aplicació està integrada correctament amb Firebase i que les notificacions push funcionen com s'espera. Confirmant que la comunicació entre l'aplicació, Firebase i els dispositius dels usuaris està establerta.

Primer, verifiquem que l'aplicació es carrega correctament des de la URL proporcionada per ngrok. A continuació, el navegador sol·licita permís per rebre notificacions, nosaltres acceptem i generem un token únic, que és una cadena de caràcters única que identifica un dispositiu o un usuari en FCM. Això es mostra a la interfície de la web (com es veu a la imatge, sota el botó Generate Token).

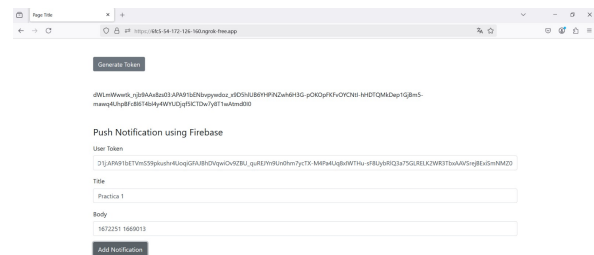


Figure 2. Web de la aplicació

Veiem que per enviar la notificació trobem 3 camps: el **User Token**, que és el token de l'usuari que rebirà la notificació. El **Title**, que és el títol de la notificació (en el nostre cas, Pràctica 1). I per últim el **Body**, que és el contingut de la notificació (en el nostre cas, són els nostres NIUS 1672251 1669013).

En clicar el botó **Add Notification**, la web envia una petició a Firebase amb el token, el títol i el cos. Firebase processa la petició i envia la notificació al dispositiu corresponent.

Si tot està configurat correctament, el dispositiu rebirà una notificació emergent amb el text introduït. Encara que l'usuari no tingui la web oberta, la notificació es mostrarà.

Seguint l'exemple de la figura 2, on l'Adrián, des de **Firefox**, envia a la Nerea, que utilitza **Chrome**, la notificació amb la informació mostrada. La Nerea rep aquesta notificació pocs instants després:

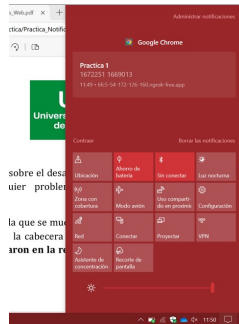


Figure 3. Notificació rebuda des de FireFox a Chrome

5.4. Un segon exemple

Vam voler provar també de fer l'enviament a la inversa, de forma que l'Adrián, des de FireFox, rebés una notificació de resposta per part de la Nerea des de Chrome. Seguint el mateix procediment de l'apartat anterior, ara la Nerea li envia un missatge a l'Adrián, saludant-lo de volta:

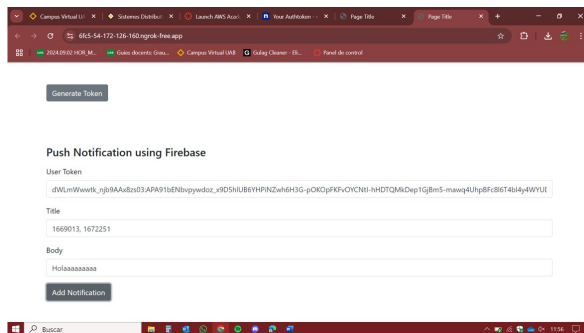


Figure 4. Notificació enviada des de Chrome a Firefox

Uns instants després, l'Adrián rep la notificació al seu navegador FireFox:

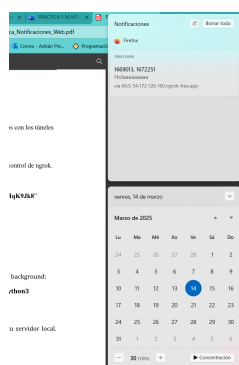


Figure 5. Notificació rebuda des de Chrome a Firefox

6. Problemes que hem trobat

Durant el desenvolupament, vam identificar diversos problemes que vam haver de solucionar pel correcte funcionament de la pràctica:

- **Comanda incorrecta:** En executar l'aplicació, la comanda `sudo /home/ec2-user/python-docs-hello-world/.venv/bin/python3`

`app.py` incloïa una ruta errònia al document. Vam corregir la ruta per `sudo /home/ec2-user/.venv/bin/python3 app.py` i l'aplicació va començar a funcionar correctament.

- **Error en l'execució de Flask:** En copiar la comanda per executar Flask, el nom del fitxer era incorrecte (`WebApp.py` en comptes de `WebApp.py`). Vam corregir el nom i l'aplicació va carregar correctament.

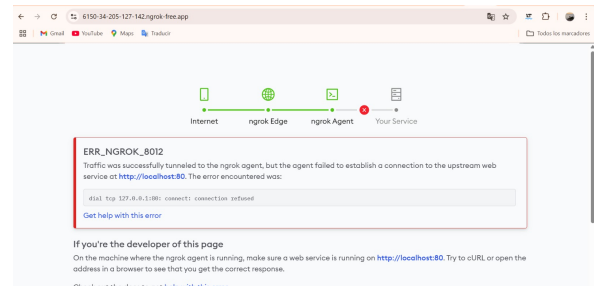


Figure 6. Pàgina que ens donava error

- **Error a Firebase:** Vam copiar per error un caràcter en la configuració de Firebase, el qual va impedir la generació del token d'usuari. Un cop corregit, tot va funcionar com s'esperava.

7. Conclusions

Aquesta pràctica ens ha permès comprendre i aplicar els conceptes fonamentals relacionats amb el desplegament d'aplicacions web al núvol, així com la integració de notificacions push amb Firebase.

En primer lloc, hem après a desenvolupar i executar una aplicació web amb Flask en un entorn local, gestionant les dependències mitjançant un entorn virtual per garantir un funcionament aïllat. Això ens ha ajudat a entendre la importància de provar les aplicacions abans de desplegar-les en un entorn de producció.

Un cop finalitzada aquesta fase, disposem d'una instància EC2 amb l'aplicació Flask en funcionament. Hem configurat l'entorn, incloent-hi les credencials AWS necessàries. A més, hem ajustat la configuració del servidor per garantir l'accessibilitat de l'aplicació des del navegador. Amb aquests passos, l'aplicació ja està desplegada al núvol i accessible públicament

Finalment, hem integrat FCM per a la gestió de notificacions push. Un cop completada, l'aplicació Flask pot enviar notificacions push als usuaris mitjançant FCM. Hem configurat correctament Firebase, establert una connexió segura amb ngrok i verificat el funcionament de les notificacions en diferents navegadors. Amb això, assegurem una comunicació en temps real entre usuaris, independentment de si tenen l'aplicació oberta, millorant així la interactivitat del sistema.